

UNIVERSITÉ SIDI MOHAMED BEN ABDELLAH

Faculté des Sciences Dhar El Mahraz — Fès

Master Big Data et Systèmes Intelligents (BDSI)

RAPPORT DE PROJET

Prédiction de la Qualité de l'Air dans les Grandes Villes Marocaines

Collecte de données réelles · Modélisation centralisée · Apprentissage fédéré

Réalisé par : BOUJAJ Imane, WARDI Safae

Filière : Master BDSI — Semestre 2

Année universitaire 2025 / 2026

Table des matières

Table des matières	2
1. Introduction et objectifs du projet	3
1.1 Cibles de prédiction	3
1.2 Architecture générale du pipeline	3
1.3 Environnement technique	3
2. Étape 1 — Collecte des données.....	4
2.1 Villes sélectionnées.....	4
2.2 Source des données et protocole de collecte.....	4
2.3 Résultats de la collecte	5
3. Étape 2 — Préparation des données	5
3.1 Nettoyage initial.....	5
3.2 Détection et interpolation des cycles incomplets	6
3.3 Reconstruction des variables temporelles et imputation finale.....	6
3.4 Encodage et sélection des variables	6
3.5 Variables cibles	7
3.6 Livrables de l'étape 2	7
4. Étape 3 — Construction des modèles.....	7
4.1 Régression — Prédiction de PM2.5	7
4.2 Classification — Prédiction de l'AQI.....	8
4.3 Visualisations produites	8
5. Volet bonus — Simulation d'apprentissage fédéré	9
5.1 Partition des données par ville	9
5.2 Implémentation manuelle de FedAvg.....	9
5.3 Implémentation avec le framework Flower.....	9
Résultats par ville (dernier round d'évaluation)	10
5.4 Synthèse du volet fédéré	10
6. Synthèse comparative des approches	11
7. Conclusion et perspectives	11
7.1 Pistes d'amélioration	12
Annexe — Architecture du projet.....	13

1. Introduction et objectifs du projet

Ce projet vise à construire un système complet de prédiction de la qualité de l'air dans plusieurs villes marocaines, en s'appuyant sur des données réelles collectées via des API publiques. La démarche couvre l'ensemble du cycle de vie d'un projet de Machine Learning : collecte de données, préparation et nettoyage, modélisation supervisée (régression et classification), puis une extension expérimentale autour de l'apprentissage fédéré (Federated Learning).

L'objectif pédagogique est double : d'une part maîtriser un pipeline de Data Science de bout en bout sur des données environnementales réelles et bruitées ; d'autre part explorer un paradigme d'apprentissage distribué dans lequel chaque ville agit comme un client détenant ses propres données, sans partage direct de celles-ci — illustrant ainsi une problématique de confidentialité proche des cas d'usage IoT et capteurs urbains.

1.1 Cibles de prédiction

- Régression : prédiction de la concentration de particules fines PM2.5 ($\mu\text{g}/\text{m}^3$).
- Classification : prédiction de la classe d'indice de qualité de l'air (AQI), sur une échelle de 1 (Bon) à 5 (Très mauvais) définie par OpenWeatherMap.

1.2 Architecture générale du pipeline

Le projet est organisé en trois étapes principales, conformes au cahier des charges, complétées par un volet bonus :

- Étape 1 — Collecte automatisée des données météorologiques et de qualité de l'air.
- Étape 2 — Préparation, nettoyage, interpolation et normalisation des données.
- Étape 3 — Construction et évaluation de modèles de régression et de classification.
- Bonus — Simulation d'apprentissage fédéré (implémentation manuelle puis via le framework Flower).

1.3 Environnement technique

Le projet est développé en Python, organisé en modules réutilisables (`src/data`, `src/models`, `src/federated`) avec une séparation claire entre la collecte, la préparation, la modélisation centralisée et la modélisation fédérée.

Composant	Outils / Bibliothèques
Collecte de données	requests, API OpenWeatherMap (Weather + Air Pollution)
Traitement des données	pandas, numpy, scikit-learn (LabelEncoder, StandardScaler, SimpleImputer)
Modélisation centralisée	scikit-learn (LinearRegression, RandomForest, DecisionTree), joblib
Visualisation	matplotlib, seaborn
Apprentissage fédéré (manuel)	Implémentation FedAvg personnalisée en NumPy / scikit-learn
Apprentissage fédéré (framework)	Flower (flwr) avec backend de simulation Ray

Tableau 1 — Pile technique utilisée dans le projet.

2. Étape 1 — Collecte des données

2.1 Villes sélectionnées

Six villes marocaines, réparties sur l'ensemble du territoire (littoral atlantique, intérieur, nord), ont été retenues afin de capturer une diversité de profils climatiques et de pollution :

Ville	Latitude	Longitude
Casablanca	33.5731	-7.5898
Rabat	34.0209	-6.8416
Marrakech	31.6295	-7.9811
Fès	34.0181	-5.0078
Tanger	35.7595	-5.8340
Agadir	30.4278	-9.5981

Tableau 2 — Coordonnées géographiques des villes étudiées.

2.2 Source des données et protocole de collecte

Les données sont récupérées via deux points d'accès de l'API OpenWeatherMap : l'endpoint météo (température, humidité, pression, vent, nébulosité) et l'endpoint qualité de l'air (AQI, PM2.5, PM10, NO2, O3, SO2, CO). Un collecteur Python dédié (collector.py) interroge ces deux API pour chaque ville, à intervalle régulier d'une heure, avec :

- Une gestion de limite de débit (rate limiting) pour respecter les quotas de l'API.

- Un mécanisme de tentatives multiples (jusqu'à 3 essais) en cas d'échec réseau.
- Une sauvegarde de point de reprise (checkpoint JSON) après chaque ville, permettant d'interrompre puis de relancer la collecte sans perte de données — un script `resume_collector.py` dédié assure cette reprise automatique.
- Une journalisation complète (logs horodatés) de chaque cycle de collecte.

La collecte était initialement planifiée sur 168 cycles horaires (soit 7 jours), pour 6 villes par cycle. En pratique, la collecte s'est étendue du 25 avril 2026 au 9 mai 2026 (environ deux semaines), des interruptions ayant nécessité plusieurs reprises grâce au mécanisme de checkpoint — illustrant la robustesse du dispositif de collecte face aux aléas réseau.

2.3 Résultats de la collecte

Ville	Enregistrements	Part
Casablanca	166	16,7 %
Rabat	167	16,8 %
Marrakech	166	16,7 %
Fès	166	16,7 %
Tanger	166	16,7 %
Agadir	165	16,6 %
Total	996	100 %

Tableau 3 — Répartition des enregistrements collectés par ville (15 échecs de requête sur l'ensemble de la collecte, compensés par le mécanisme de reprise).

Au total, 996 enregistrements bruts ont été collectés, répartis sur 21 colonnes (identifiants, coordonnées, variables météorologiques et variables de qualité de l'air). Chaque enregistrement correspond à une mesure horodatée pour une ville et un cycle de collecte donnés.

3. Étape 2 — Préparation des données

3.1 Nettoyage initial

- Conversion de la colonne timestamp au format date/heure.
- Suppression de 3 enregistrements dupliqués (clé `cycle_index` + ville).
- Aucune valeur manquante directe détectée sur les colonnes brutes après dédoublement.

3.2 Détection et interpolation des cycles incomplets

L'analyse par cycle de collecte a révélé 3 cycles incomplets, où une ville n'avait pas pu être interrogée avec succès :

- Cycle 7 : 5/6 villes (Casablanca manquante).
- Cycle 18 : 5/6 villes (Agadir manquante).
- Cycle 36 : 5/6 villes (Casablanca manquante).

Pour corriger ces trous, une grille complète de 166 cycles × 6 villes a été reconstruite (index multiple cycle/ville), puis une interpolation linéaire a été appliquée par ville sur les variables numériques (température, humidité, pression, vent, polluants). Cette opération a permis de combler 39 valeurs manquantes, portant le jeu de données à 996 lignes complètes, soit 166 cycles intégralement renseignés sur 166 (100 %).

3.3 Reconstruction des variables temporelles et imputation finale

Les variables calendaires (heure, jour de la semaine, jour du mois, mois) ont été recalculées à partir du timestamp consolidé, afin de garantir leur cohérence après interpolation. Les quelques valeurs résiduelles (4 colonnes temporelles) ont ensuite été imputées par la moyenne via un SimpleImputer, ramenant le nombre de valeurs manquantes à zéro sur l'ensemble du jeu de données.

3.4 Encodage et sélection des variables

La variable catégorielle « ville » a été encodée numériquement via un LabelEncoder :

Ville	Code
Agadir	0
Casablanca	1
Fès	2
Marrakech	3
Rabat	4
Tanger	5

Tableau 4 — Mapping d'encodage des villes (city_encoded).

Douze variables explicatives ont été retenues pour la modélisation : temp, humidity, pressure, wind_speed, pm10, no2, o3, so2, co, hour, day_of_week et city_encoded. Neuf d'entre elles (les variables numériques continues) ont été normalisées par standardisation (StandardScaler, moyenne 0 / écart-type 1).

3.5 Variables cibles

Deux cibles ont été préparées en parallèle :

- PM2.5 (régression) — variable continue, 560 valeurs uniques observées.
- AQI (classification) — variable catégorielle, 3 classes effectivement représentées dans les données collectées.

Classe AQI	Libellé	Effectif	Part
1	Bon	39	3,9 %
2	Moyen	621	62,3 %
3	Dégradé	336	33,7 %

Tableau 5 — Distribution des classes d'AQI dans le jeu de données préparé. Les classes 4 (Mauvais) et 5 (Très mauvais) ne sont pas représentées sur la période de collecte, et la classe 1 (Bon) reste minoritaire.

3.6 Livrables de l'étape 2

- data/processed/air_quality_clean.csv — jeu de données nettoyé, non normalisé.
- data/processed/air_quality_scaled.csv — jeu de données normalisé, prêt pour la modélisation.
- models/federated/city_encoder.pkl et feature_scaler.pkl — encodeurs sauvegardés pour une réutilisation cohérente (notamment côté apprentissage fédéré).

4. Étape 3 — Construction des modèles

4.1 Régression — Prédiction de PM2.5

Le jeu de données a été divisé en un ensemble d'entraînement (796 échantillons, 80 %) et un ensemble de test (200 échantillons, 20 %). Trois modèles ont été entraînés et comparés sur les douze variables explicatives sélectionnées.

Modèle	MAE ($\mu\text{g}/\text{m}^3$)	RMSE ($\mu\text{g}/\text{m}^3$)	R ²
Régression Linéaire	0,839	1,118	0,802
Random Forest	0,328	0,463	0,966
Arbre de Décision	0,498	0,763	0,908

Tableau 6 — Comparaison des modèles de régression sur l'ensemble de test (meilleur modèle surligné).

Le modèle Random Forest se distingue très nettement, avec un coefficient de détermination $R^2 = 0,966$ et une erreur absolue moyenne inférieure à $0,33 \mu\text{g}/\text{m}^3$. Sa capacité à capturer des relations non linéaires entre les polluants co-occurents (PM10, NO2, O3...) et la cible PM2.5 explique cet écart de performance par rapport à la régression linéaire, plus contrainte. L'arbre de décision unique, plus simple que la forêt aléatoire, obtient un résultat intermédiaire ($R^2 = 0,908$), avec un risque de sur-apprentissage plus marqué qu'un ensemble de plusieurs arbres.

4.2 Classification — Prédiction de l'AQI

Pour la classification, un découpage stratifié 80/20 a été utilisé afin de préserver la proportion des classes dans les ensembles d'entraînement et de test, compte tenu du déséquilibre observé (62,3 % de la classe « Moyen »). Un classifieur Random Forest a été entraîné sur les mêmes douze variables explicatives.

Classe	Précision	Rappel	F1-score	Support
1.0 (Bon)	1,00	0,62	0,77	8
2.0 (Moyen)	0,98	1,00	0,99	125
3.0 (Dégradé)	1,00	1,00	1,00	67
Moyenne pondérée	0,99	0,98	0,98	200

Tableau 7 — Rapport de classification détaillé (Random Forest Classifier) — Accuracy globale : 98,5 % · F1-score pondéré : 0,983.

Les performances globales sont excellentes (98,5 % d'exactitude). Le seul point d'attention concerne la classe minoritaire « Bon » (1,0), pour laquelle le rappel chute à 0,62 : sur les 8 échantillons de test de cette classe, plusieurs sont confondus avec la classe « Moyen » voisine. Ce comportement est cohérent avec la faible représentation de cette classe dans les données (3,9 % des observations) et illustre une limite classique des modèles entraînés sur des distributions déséquilibrées — une piste d'amélioration possible serait l'utilisation de techniques de rééquilibrage telles que SMOTE.

4.3 Visualisations produites

- model_comparison.png — comparaison MAE/RMSE/ R^2 des trois modèles de régression.
- confusion_matrix.png — matrice de confusion du classifieur AQI.
- feature_importance.png — importance des variables dans le modèle Random Forest (régression).
- error_distribution.png — distribution des erreurs de prédiction du modèle Random Forest.

5. Volet bonus — Simulation d'apprentissage fédéré

L'apprentissage fédéré (Federated Learning) permet d'entraîner un modèle global sans centraliser les données : chaque ville agit comme un client détenant ses propres mesures, entraîne un modèle local, puis ne transmet que les paramètres appris (coefficients, biais) à un serveur central chargé de les agréger. Cette approche a été explorée selon deux implémentations complémentaires.

5.1 Partition des données par ville

Le jeu de données normalisé a été partitionné par ville, chacune des 6 villes constituant un client fédéré indépendant, avec un volume d'environ 166 échantillons par client.

5.2 Implémentation manuelle de FedAvg

Une première implémentation « from scratch » reproduit l'algorithme Federated Averaging (FedAvg) avec un modèle de régression linéaire : à chaque round, chaque client entraîne un modèle local sur ses données, puis le serveur calcule une moyenne pondérée des coefficients (pondération proportionnelle au nombre d'échantillons de chaque client) pour mettre à jour le modèle global. Cinq rounds ont été simulés.

Modèle	MAE	RMSE	R ²
Centralisé — Random Forest	0,328	0,463	0,966
Centralisé — Régression Linéaire	0,839	1,118	0,802
Fédéré — Régression Linéaire (FedAvg, 5 rounds)	1,481	1,929	0,412

Tableau 8 — Comparaison du modèle fédéré manuel avec les modèles centralisés.

Comparé au Random Forest centralisé, l'écart est important (R^2 centralisé – R^2 fédéré = 0,554), mais cette comparaison n'est pas équitable puisque les deux modèles diffèrent par nature (ensemble d'arbres contre régression linéaire simple). Une comparaison à architecture égale (régression linéaire centralisée : R^2 = 0,802 contre régression linéaire fédérée : R^2 = 0,412) confirme néanmoins un écart significatif, plus marqué qu'attendu. L'explication principale réside dans l'hétérogénéité des données entre villes (configuration non-IID) : la moyenne des coefficients locaux ne représente pas nécessairement un optimum global pertinent lorsque les relations entre variables diffèrent sensiblement d'une ville à l'autre.

5.3 Implémentation avec le framework Flower

Afin d'aller au-delà de la simulation manuelle, le projet a été étendu avec le framework open-source Flower (flwr), qui fournit une architecture client/serveur standardisée pour l'apprentissage fédéré, ici exécutée en simulation locale via le backend Ray. L'architecture comprend trois modules :

- `task.py` — chargement des données, partition par ville, construction du modèle (régression linéaire).
- `client_app.py` — définition du client fédéré (`AirQualityClient`), implémentant `get_parameters`, `set_parameters`, `fit` et `evaluate`.
- `server_app.py` — configuration de la stratégie serveur (FedAvg natif de Flower, agrégation pondérée, minimum de 3 clients pour l'entraînement et l'évaluation).

La simulation a été exécutée sur 10 rounds avec 6 clients actifs (un par ville), Flower attribuant un identifiant de partition à chaque client simulé. Une difficulté technique a été rencontrée lors de la première exécution : le chemin d'accès au fichier de données était relatif au répertoire d'exécution et provoquait une erreur `FileNotFoundError` dans l'environnement distribué de Ray. Le problème a été résolu en construisant un chemin absolu vers la racine du projet dans `task.py`, garantissant un accès fiable au fichier quel que soit le worker Ray exécutant le client.

Résultats par ville (dernier round d'évaluation)

Ville	MAE	R ²
Agadir	0,944	0,783
Tanger	0,950	0,771
Marrakech	1,148	0,698
Rabat	2,430	0,463
Fès	1,488	-0,074
Casablanca	2,754	-0,399

Tableau 9 — Performance du modèle global fédéré, évaluée localement sur les données de test de chaque ville.

Ces résultats mettent clairement en évidence la non-homogénéité des relations climat/pollution selon les villes : le modèle global fonctionne bien pour Agadir, Tanger et Marrakech (R^2 entre 0,70 et 0,78), reste moyen pour Rabat ($R^2 = 0,46$), et devient inadapté pour Fès et Casablanca (R^2 négatif, signifiant que le modèle fédéré prédit moins bien que la simple moyenne locale). Sur l'ensemble des 10 rounds, la perte distribuée globale (history loss) reste quasi stable autour de 1,605, ce qui traduit une convergence limitée du modèle linéaire fédéré face à cette hétérogénéité — un comportement attendu compte tenu de la simplicité du modèle agrégé et du faible nombre de rounds.

5.4 Synthèse du volet fédéré

- FedAvg permet d'entraîner un modèle global performant sans jamais transférer les données brutes des villes, répondant à une logique de confidentialité par conception.

- La performance du modèle fédéré dépend fortement de l'homogénéité (caractère IID) des données entre clients ; ici, les profils climatiques et de pollution contrastés des villes marocaines limitent les gains du modèle global unique.
- Le passage d'une implémentation manuelle à un framework dédié (Flower) confirme les mêmes tendances tout en offrant une infrastructure plus robuste, prête pour un passage à l'échelle (davantage de clients, de rounds, ou de stratégies d'agrégation comme FedProx).

6. Synthèse comparative des approches

Approche	MAE	RMSE	R ²
Random Forest (centralisé)	0,328	0,463	0,966
Arbre de Décision (centralisé)	0,498	0,763	0,908
Régression Linéaire (centralisée)	0,839	1,118	0,802
Régression Linéaire (fédérée, FedAvg manuel)	1,481	1,929	0,412

Tableau 10 — Vue d'ensemble des performances obtenues (tâche de régression PM2.5).

Cette synthèse confirme que, pour un jeu de données de cette taille centralisable sans contrainte réglementaire forte, le Random Forest centralisé reste le choix le plus performant. L'apprentissage fédéré démontre néanmoins sa pertinence conceptuelle pour des scénarios où la centralisation des données est impossible ou indésirable (confidentialité, souveraineté des données municipales, contraintes réseau), au prix d'une perte de performance directement liée à l'hétérogénéité des données locales.

7. Conclusion et perspectives

Ce projet a permis de mettre en œuvre un pipeline complet de Data Science appliqué à la qualité de l'air au Maroc, depuis la collecte automatisée de données réelles via API jusqu'à la comparaison de plusieurs paradigmes de modélisation. Les résultats obtenus sont très satisfaisants pour la modélisation centralisée, avec un Random Forest atteignant un R² de 0,966 en régression et une exactitude de 98,5 % en classification.

L'extension vers l'apprentissage fédéré, à la fois en implémentation manuelle et via le framework Flower, a permis d'illustrer concrètement les compromis entre confidentialité des données et performance du modèle, ainsi que la sensibilité de FedAvg à l'hétérogénéité (non-iidité) des données entre clients — une problématique centrale et bien documentée dans la littérature sur l'apprentissage fédéré.

7.1 Pistes d'amélioration

- Étendre la période de collecte pour disposer de davantage de cycles et de classes AQI plus rares (« Mauvais », « Très mauvais »).
- Tester des stratégies d'agrégation plus robustes à l'hétérogénéité, telles que FedProx ou FedAvgM.
- Expérimenter des modèles fédérés non linéaires (par exemple, agrégation d'arbres ou de réseaux de neurones légers) pour réduire l'écart avec le Random Forest centralisé.
- Appliquer des techniques de rééquilibrage de classes (SMOTE) pour améliorer le rappel sur la classe AQI « Bon ».
- Envisager une personnalisation par client (federated transfer learning) afin de mieux exploiter les spécificités locales mises en évidence pour Casablanca et Fès.

Annexe — Architecture du projet

Arborescence simplifiée du dépôt de projet :

```
projet_air_quality/
├── data/
│   ├── raw/                # Fichiers CSV bruts par campagne de collecte
│   ├── processed/          # air_quality_clean.csv, air_quality_scaled.csv
│   └── checkpoints/        # collection_progress.json (reprise automatique)
├── docs/                   # Rapport, slides, supports de présentation
├── logs/                   # Journaux de collecte horodatés
├── models/
│   ├── centralized/        # random_forest_regressor.pkl, classifier.pkl,
│   │                       # linear_regression.pkl
│   └── federated/          # city_encoder.pkl, feature_scaler.pkl, federated_model.pkl
├── src/
│   ├── data/               # collector.py, config.py, data_preparation.py, monitor.py
│   ├── models/             # model_building.py
│   └── federated/
│       ├── federated_manual.py # Implémentation FedAvg manuelle
│       └── flower/          # client_app.py, server_app.py, task.py, pyproject.toml
└── visualizations/         # Graphiques générés (PNG)
```